Who Backs Into the Bay? Benchmarking Negotiation Policies for Head-On Robot Encounters in Single-Lane Corridors

Brian Deegan

***Abstract*—When two autonomous mobile robots meet head-on in a corridor that admits only one, one of them must reverse into a pull-in bay or out of the corridor. We study who should yield, and into which refuge, with a benchmark that runs the same agent code in a 2D unicycle simulator and in ROS 2/Gazebo with TurtleBot3 Waffle robots. We compare a non-communicating reactive baseline, fixed priority, an ordered rulebook, first-come corridor reservation, cost bidding, a group bid for convoys, a Clarke-pivot mechanism, and a learned tabular negotiator against the best forced yielder, a hindsight reference that tries both yield decisions. In 2 940 single-bay episodes the reactive baseline succeeds in 93.8% and incurs 3.6× the reference's total delay; the negotiating policies succeed in every episode. Cost bidding with exact costs matches the reference; with unequal urgency its urgency-weighted excess delay is 0.04 s against 27.1 s for fixed priority, and with log-normal bid noise of $\sigma = 0.5$ it is no better than a noise-free closest-refuge rule. Lower-bid-yields bidding can be gamed; with Clarke-pivot payments no unilateral misreport we tested pays off under the predicted costs, at the price of a budget surplus and no protection against collusion. Reservation is slower than fixed priority in our layout. Under 90–99% packet loss, a factorial ablation of three protocol safeguards (5 760 episodes) observed no collisions in any arm. We extend the benchmark to convoys, a T-junction, inconsistent and wrong maps, and exploring robots that share their map only after sensing each other, which helps only if detection precedes the decision. In 460 Gazebo runs on the same code, a reduced grid over every study, no run outside the reactive baseline failed or collided, and 437 of those 450 runs chose the same first yielder as 2D. Code and per-episode results are released.**

***Index Terms*—Multi-robot systems, negotiation, deadlock, path planning for multiple mobile robots, benchmarking.**

I. Introduction

NARROW aisles, doorways, and service corridors force autonomous mobile robots (AMRs) into conflicts that local collision avoidance cannot resolve: if two robots meet head-on where only one fits, someone must back off. Reciprocal collision avoidance and control-barrier-function controllers deadlock in such symmetric configurations [1], [2], [3], while centralized planners and traffic managers avoid the conflict at the price of a global plan [4], [5], [6], [7], [8], [9]. Between these extremes lies decentralized negotiation: the robots exchange messages and agree who yields. Rules of the road [10], [11], auctions [12], [13], learned policies [14], [15], language-model negotiation [16], [17], and foundation-model high-level planners for deadlock resolution [18] have all been proposed, but each paper builds its own scene and metrics, and no study compares the families under the same physics and communication model [19].

This letter contributes:

1) an open benchmark of the head-on corridor encounter with pull-in bays, in which the same agent state machine runs in a 2D simulator for Monte Carlo sweeps and in ROS 2 Jazzy/Gazebo Harmonic with TurtleBot3 Waffles;
2) a comparison of reactive, fixed-priority, rule-based, reservation, cost-bidding, group-bidding, Clarke-pivot, and learned negotiators over bay geometry, robot asymmetry, bid noise, strategic misreporting, repeated encounters, and packet loss and latency, all on one code version with per-episode results;
3) a factorial ablation of the protocol safeguards under extreme packet loss, and extensions to convoys, a T-junction, inconsistent maps, physically blocked bays, and exploring robots that share their map only after detecting each other;
4) a Gazebo check of where the 2D conclusions transfer: 460 runs on the final code, paired by seed with 2D, over a reduced grid of every study.

II. Related Work

Deadlock in narrow passages. Symmetric encounters are a known failure mode of decentralized avoidance. Grover et al. [1] characterize CBF deadlocks; Dergachev and Yakovlev [2] switch to local multi-agent path finding (MAPF) when ORCA stalls; Chandra et al. [3] perturb controls onto liveness sets in doorways and hallways. These works resolve who goes first by slowing down, not by choosing which robot reverses into which refuge. *Centralized coordination.* MAPF and AGV traffic management treat bays as graph nodes [20], [5], [21], [22]; safe-interval planning reserves corridor segments in time [6], fleet managers such as Open-RMF negotiate conflicting schedules centrally [9], [8], and precedence-preserving coordinators provide safety under delays [4], [23]. They require a global planner or schedule database and typically charge reversing as ordinary motion. *Negotiation and mechanisms.* Social laws [11], market-based allocation [12], and negotiated path planning [13] assume truthful costs. Vickrey–Clarke–Groves mechanisms [24], [25], [26], [27] make truthful reporting dominant, but no efficient, truthful mechanism is budget balanced in general [28], [29]. Karma schemes trade non-monetary credit across repeated

B. Deegan is with Electrical and Electronic Engineering, University of Galway, Ireland (e-mail: brian.deegan@universityofgalway.ie).

DRAFT, not for distribution.

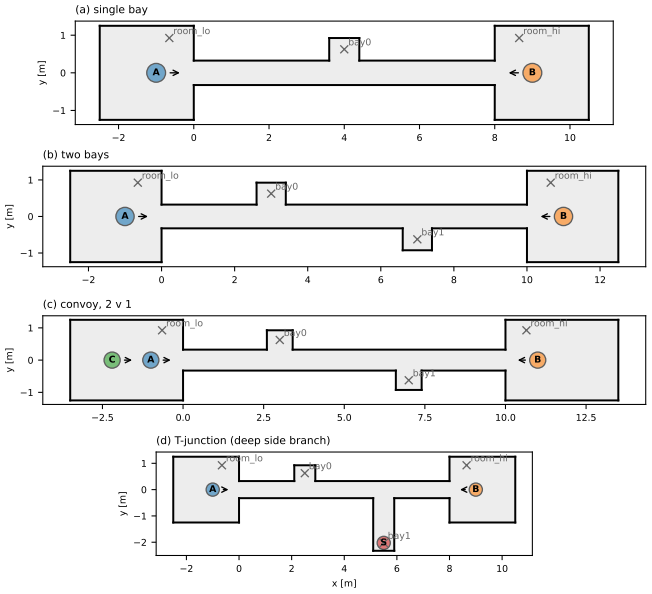


Fig. 1. Benchmark scenarios (gray: free space; ×: refuge parking poses). Arrows show travel direction; in (d) robot S starts in the side branch and A's destination is the branch.

encounters to make right-of-way allocation fair [30], [31]; cooperative give-way at road bottlenecks has also been studied for connected vehicles [32]. *Learning and LLMs.* MARL give-way tasks exist in VMAS [14]; MARLIN uses LLM negotiation to guide MARL training in narrow-corridor swaps, with physical TurtleBot3 experiments [16]; GameChat sets priority through LLM dialogue in doorways [17]. None compares against rule-based, bidding, and hindsight baselines under communication impairment, which is the gap we address.

III. Benchmark

A. Scenarios

The base scenario (Fig. 1a) is an 8 m×0.65 m corridor between two 2.5 m rooms. A TurtleBot3 Waffle (0.25 m swept radius, 0.26 m/s forward, 0.15 m/s reverse) fits; two cannot pass. Bays are 0.8 m wide and 0.6 m deep, with configurable position, side, and number; the end rooms also serve as refuges. Robot A enters at $t = 0$, and B's start delay is drawn from $U[0, 8]$ s. Variants add a second bay (Fig. 1b), a two-robot convoy against one oncoming robot (Fig. 1c), and a T-junction (Fig. 1d) whose side branch is approximated as a 2 m-deep bay that a robot may leave or have as its destination; it is not a full 2D junction. Robots can differ in urgency, payload, and speed limits. Communication is a broadcast bus with configurable latency, jitter, and loss.

B. Agent and protocol

Each robot runs the same state machine (cruise, negotiate, yield, proceed, done) at 20 Hz, with lidar perception and a unicycle model. A conflict is detected when an oncoming peer is within the detection range (6 m by default). Robots broadcast STATE messages at 5 Hz and, on conflict, exchange BID messages containing a snapshot of their pose, speed, urgency, payload, and known refuges. Both robots then run the same deterministic decision function on the same pair of snapshots, so they agree without a third message when no message is lost. A yield plan names the yielder and its refuge; feasibility checks that the refuge lies behind the yielder and is not taken.

With message loss the robots may decide on different snapshots. Three safeguards address this, and each can be switched off for ablation. *Commit*: each robot broadcasts its decision and acts only once it holds the peer's, the lower-ID robot's decision wins a disagreement, and an unconfirmed decision expires after a timeout. *State guard*: a yielder whose peer's STATE shows it yielding into the same refuge, or yielding to it, gives way and proceeds instead. *Bay check*: a bay that the lidar shows occupied counts as taken, and a yielder that finds a robot in its bay proceeds. A robot waiting to pass also renegotiates if its supposed yielder approaches it. We give no formal safety or liveness proof for this protocol; its behavior is characterized empirically in Sec. IV-C.

C. Policies

Reactive: no messages; a blocked robot waits a random patience time and then backs to the nearest refuge behind it. *Fixed priority*: the higher-ID robot always yields. *Rulebook*: an ordered list of rules (no feasible refuge; urgency; payload; first in corridor; refuge within 2 m behind; ID tie-break), the first discriminating rule decides; a variant puts "closest refuge yields" first. *Reservation*: first come, first served; a robot still outside the corridor while an oncoming robot is inside waits in its end room, and robots that are both inside fall back to the rulebook. `cost_bid`: for each candidate yielder Y with proceeder P, both robots predict the delays d_Y, d_P of Y's best feasible plan from the robot kinematics and evaluate $u_Y d_Y + u_P d_P$, where u is urgency; the cheaper outcome is chosen. `group_bid`: as `cost_bid`, but a side's cost includes each yielding member's own urgency-weighted delay, the wait of the yielding side while the other side passes, and the delay of the proceeding side's followers. *Clarke*: each robot reports its predicted urgency-weighted delay $r_i(o)$ under both outcomes o; the mechanism chooses $o^* = \arg\min_o \sum_i r_i(o)$ and charges robot i the Clarke pivot $p_i = \sum_{j\neq i} r_j(o^*) - \min_o \sum_{j\neq i} r_j(o)$ in delay credit [25]. *Learned*: a tabular contextual bandit over a coarse observation (each robot's binned distance to its nearest feasible refuge behind it, which robot is more urgent, and both payload flags), trained on 800 encounters by running both forced outcomes; unseen observations fall back to fixed priority. It is a lightweight learned baseline, not deep multi-agent RL.

D. Metrics and statistics

We report success (all robots reach their goals before the time limit without collision), deadlock (time limit reached), collision (overlap of the Waffle footprint rectangles), total delay relative to free-flow travel time, urgency-weighted delay, reverse distance, and Jain's fairness index [33].

Excess delay. The delay of an episode is $D = \sum_i w_i\,(T_i - T_i^{\mathrm{free}})$, where T_i is robot i's travel time, T_i^{free} its travel time

with the corridor to itself, and $w_i = 1$ (total delay) or w_i equal to the robot's urgency (weighted delay). For every two-robot encounter k we also run the two forced outcomes, A yields and B yields, each into that robot's best feasible refuge; the better of the two is the *best forced yielder*. The excess delay of policy π over N encounters is

$$\Delta_\pi = \frac{1}{N}\sum_{k=1}^{N}\Big(D_\pi^{(k)} - \min\big(D_A^{(k)}, D_B^{(k)}\big)\Big), \qquad (1)$$

paired by encounter, with the same weights w_i in all three terms. $\Delta_\pi = 0$ means the policy always made the better of the two yield decisions. The reference chooses only who yields, so Δ_π measures the quality of that decision, not the distance to a globally optimal joint motion plan, and a policy that also chooses *when* to move could in principle have $\Delta_\pi < 0$. It is the ex-post regret of decision theory; we avoid that term because it has different meanings in online learning and in mechanism design. Excess delay is defined for two-robot encounters; for the convoy we add a side-level reference, the better of "the convoy yields" and "the lone robot yields".

Statistics. All 2D results come from one code version, and every per-episode result is released with the scripts that produce the tables. A seed fixes the start delays and the communication noise; at 0% loss the negotiating policies are deterministic given the start delays, so the independent samples are the distinct draws, and policies are compared on the same draws. Delays are averaged over successful episodes only; failed episodes are censored at the time limit and reported through the success, deadlock, and collision rates. $\pm$ values are 95% confidence intervals of the mean; because Δ_π is a per-encounter paired difference, its interval is a paired interval. Rates carry Wilson 95% intervals, and an observed count of zero is reported with its 95% upper bound.

IV. Results

A. Two robots, one corridor

Price of no communication. Over 2 940 single-bay episodes (60 draws at each of 7 bay positions) the reactive baseline succeeded in 93.8% [91–96] of episodes; the rest deadlocked. Its total delay over successes was 97 ± 5 s against 26.8 ± 0.6 s for the best forced yielder, it reversed 5.7 m against 0.64 m, and both robots backed off in 39% of episodes. All negotiating policies succeeded in all 420 of their episodes without collision.

Price of decentralization and bay position. With symmetric robots, fixed priority and the default rulebook make identical decisions, with 6.4 ± 0.6 s excess delay, rising to 14.6–14.8 s when the bay is 2–3 m from A's entrance and the late robot B is sent back past it. Putting "closest refuge yields" first reduces the excess delay to 3.1 ± 0.5 s. `cost_bid` chose the reference's yielder in every single- and two-bay episode. This match is a best case, because the bids use the simulator's own kinematics.

Reservation. Over 210 encounters (bay at 1–7 m, 30 draws each), first-come reservation succeeded in every episode but had 18.8 ± 0.9 s excess delay, against 6.5 ± 0.9 s for fixed priority and 0 for `cost_bid`, and it beat the best forced yielder in none of the 210 encounters. In this layout B starts on A's exit path, so waiting for the corridor means pulling into its end room rather than stopping at the entrance; layouts with a holding point outside the exit path would favor reservation.

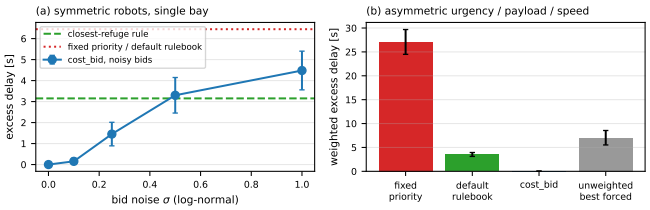


Fig. 2. (a) Excess delay (1) of `cost_bid` over the best forced yielder as bid noise grows, compared with two noise-free rulebooks. (b) Urgency-weighted excess delay with asymmetric robots. Bars and markers are means; error bars are 95% confidence intervals. Values are computed from the released per-episode results.

Noisy bids. Multiplying each bid by a log-normal factor $e^{\sigma\epsilon}$, $\epsilon \sim \mathcal{N}(0,1)$, raises `cost_bid`'s excess delay to 0.2 ± 0.1, 1.5 ± 0.6, 3.3 ± 0.8, and 4.5 ± 0.9 s at $\sigma = 0.1, 0.25, 0.5, 1$ (Fig. 2a). At $\sigma = 0.5$ bidding is no better than the noise-free closest-refuge rule (3.2 ± 0.8 s on the same draws); it is clearly ahead only for $\sigma \le 0.25$.

Asymmetric robots. With differing urgency (1 or 3), payload, or half speed (5 040 episodes), the urgency-weighted excess delay is 27.1 ± 2.6 s for fixed priority, 3.5 ± 0.4 s for the default rulebook, and 0.04 ± 0.02 s for `cost_bid`, which picked the weighted reference's yielder in 98% of episodes (Fig. 2b). The reference that minimizes unweighted delay has 7.0 ± 1.5 s weighted excess delay: the choice of objective matters as much as the protocol.

B. Incentives and fairness

Misreporting under lower-bid-yields. If B inflates its bid by $2\times$ or more, it never yields: its own delay falls from 9.1 ± 1.4 to 8.1 ± 1.2 s while total delay rises by 4.9 s (+20%). Lower-bid-yields bidding is not incentive compatible, and an earlier payment rule we implemented, in which the proceeder paid the yielder's reported bid, was also gameable (A gained 7.5 s by reporting $0.25\times$ its cost, B 0.7 s at $0.75\times$).

Clarke-pivot payments. We scaled one or both robots' reports by $k \in \{0.25, \ldots, 4\}$ (10 values) over 3 bay positions, B urgency 1 and 3, and 60 draws (10 080 episodes, all successful). No unilateral misreport by A or B increased the deviator's utility, its delay plus payment, under the predicted costs; under the realized simulator delays the largest gain was 0.02 s. This is dominant-strategy truthfulness with respect to the shared cost model only. The mechanism is not budget balanced, consistent with [28], [29]: the bank collects 9.3 s (B urgency 1) and 10.5 s (urgency 3) of credit per encounter, which a fleet would have to recycle as a virtual currency. Joint shading is profitable: if both robots report $0.5\times$, their combined utility improves by 4.9 s. Payments are charged in credit, so a robot that expects a large payment may prefer not to participate; we do not analyze individual rationality.

Fairness. Fixed priority makes B yield in every episode. Over 50 sequences of 20 repeated encounters, cost-optimal

TABLE I
PROTOCOL ABLATION UNDER 90–99% LOSS AND 2–8 S LATENCY (360 EPISODES PER ROW; THE BAY CHECK HAD NO EFFECT AND IS OMITTED). NO ROW HAD A COLLISION.

Policy	Commit	State	Success [%]	Deadl.	Delay [s]
Fixed priority	✓	✓	99.2	3	89.8 ± 3.9
	✓	–	99.2	3	90.0 ± 4.0
	–	✓	98.1	7	86.0 ± 3.6
	–	–	98.6	5	86.8 ± 3.7
`cost_bid`	✓	✓	98.9	4	79.6 ± 3.9
	✓	–	99.2	3	79.9 ± 3.9
	–	✓	99.4	2	76.4 ± 3.7
	–	–	99.7	1	76.8 ± 3.7

State: state guard. Deadl.: deadlocks. Delay: mean total delay over successful episodes.

choice already reaches a Jain index of 0.93 on cumulative delays; fixed priority costs 27% more delay for 0.75, strict alternation 22% for 0.99, and credit-weighted choice, which minimizes total delay plus λ times the difference in cumulative delays, reaches 0.997 at $\lambda = 0.25$ for 8% extra delay. This is a simple instance of the credit schemes of [30], [31].

C. Communication impairment

With 6 m detection, loss up to 90%, and latency up to 2 s (3 600 episodes), and with 1.5 m detection and robots at 0.5 m/s (4 320 episodes), every policy succeeded in at least 99.3% of its episodes with no collisions. Under extreme conditions (loss 90–99%, latency 2–8 s, 1 440 episodes) an earlier version of the protocol collided in 6 episodes, each after both robots yielded on inconsistent decisions. On the current code the default protocol has no collisions in these 1 440 episodes and succeeds in 98.9–99.2% per policy.

Table I ablates the three safeguards on the same extreme grid (360 episodes per arm). No arm collided, including the one with all safeguards off (95% upper bound 1.1% per arm), so on the current code we cannot attribute collision avoidance to any single safeguard; the remaining protection comes from the yield and pass logic itself. The lidar bay check never triggered, and its arms were identical episode for episode to those without it. Commit improved fixed priority's success from 98.1–98.6% to 99.2% at 3–4 s extra delay, but did not help `cost_bid`. In 14–24% of episodes both robots started a yield before one gave way.

D. Three robots and junctions

Convoy. With a two-robot convoy against one oncoming robot (72 runs per cell, 0% and 50% loss), a yielding leader could be blocked by its own follower; a "make room" rule, in which the follower backs into its own refuge, removed these deadlocks, and every policy then succeeded in 72/72 runs. With equal urgency, pairwise `cost_bid` ignores the follower and averaged 55.6 s total delay; `group_bid` reduced this to 34.4 s, close to fixed priority and the side-level reference (both 33.7 s). When the lone robot is urgent ($u = 3$), the side-level reference has 85.7 s urgency-weighted delay, fixed priority 90.3 s, `group_bid` 104.7 s, and `cost_bid` 117.1 s: `group_bid` still underestimates what a convoy yield costs.

T-junction. With one robot turning into the branch, one passing, and one leaving the branch (48 runs per cell), naive waiting in the lane deadlocked fixed priority in 19 of 48 runs at 0% loss. Letting the turning robot wait in a free bay before the junction gave 48/48 for fixed priority, the rulebook, and `cost_bid` at 0% and 50% loss, with mean delay over successes of 60.5, 53.1, and 48.9 s at 0% loss. The reactive baseline succeeded in 28/48. All failures in this study were deadlocks.

E. Inconsistent and wrong maps

When robots know different bays (two-bay corridor, 10 seeds per cell), map disagreement caused no collision: commit detects the mismatch and both adopt the lower-ID robot's decision. It costs time when that robot has the poorer map, +9.1 s for `cost_bid` and +29.6 s for the rulebooks; listing known bays in each BID removed the cost in every case tested. When the useful bay was physically blocked, every negotiating policy initially deadlocked (10/10), because the yielder's bay check mistook the obstacle for a robot. Distinguishing obstructions from robots via STATE messages and falling back to the next refuge restored 10/10 success with no collisions, at about +46 s because the obstruction is discovered only on arrival. These samples are small.

F. Exploring robots

In explore mode a robot starts knowing only the corridor and its end rooms and adds a bay to its map once the bay is within lidar range (3.5 m). This is geometric discovery on a known layout, not SLAM. With share-on-detect, a robot sends its known bays only after its own lidar has detected the peer; STATE and BID messages flow as before. In the two-bay corridor (1 440 episodes, 30 seeds per cell) every episode succeeded without collision. At the default 6 m negotiation range, not sharing cost $+29.2$ s for fixed priority and the rulebook and $+8.8$ s for `cost_bid` against the full map, because each robot had seen only its nearest bay when they decided; sharing the map in every BID removed the cost. Share-on-detect equalled not sharing at 6 m: the robots decide before either lidar can see the other, so the map never arrived in time (0/30). When negotiation started only at 3.5 m, share-on-detect matched the full map in 2D. In Gazebo it did not: detection from lidar clusters first occurred 2.3–2.6 m apart on average, after the decision, so the map again arrived late (0/6) and fixed priority lost 6.8 s. Sharing on detection therefore helps only if detection reliably precedes the decision.

We therefore added an opt-in hold mode (`hold_for_map`, off by default). A share-on-detect robot postpones its decision, still driving its lane and refreshing its BID snapshot, until its own lidar has detected the peer and it has the peer's map (a MAP message, or a BID carrying the map). After a 15 s timeout it decides as before. In 2D (360 extra episodes at 0 and 50% loss) every hold episode succeeded, with no collision and no timeout. At 6 m, holding removed share-on-detect's excess delay ($+29.2$ s for fixed priority, $+8.8$ s for `cost_bid`), ending 0.4–0.5 s below the full map, at the cost of 4.9–5.5 s from conflict to decision instead of 0.1–0.3 s. At

3.5 m it changed little ($+0.1$ s against $+1.7$ s for fixed priority at 50% loss). In Gazebo (24 hold runs, seeds 0–5, 0% loss) every run succeeded without collision or timeout, and the yielder used the nearer bay in all of them, including the seed where share-on-detect had sent fixed priority's yielder to its end room. But the decision came 4.7 s (3.5 m) and 9.5 s (6 m) after the conflict began, because the lidar first sees the peer only 2.3–2.6 m away, and each run took 4.7–4.9 s longer than the paired 2D run. Against the Gazebo full-map run of the same seed, total delay was $+10.0$ s for both policies, more than share-on-detect without holding ($+7.0$ s for fixed priority, $+0.3$ s for `cost_bid`). In 5 of 6 seeds each robot had already sensed both bays itself when it detected the peer, so in Gazebo the hold works by waiting, not through the exchanged map. Holding thus fixes the wrong-refuge failure but not its cost; sharing the map in every BID remains the cheaper fix in both simulators.

TABLE II
GAZEBO RUNS ON THE FINAL CODE, EACH PAIRED WITH A 2D RUN ON THE SAME SEED.

Scenario	Runs	Succ.	Coll.	Same	ΔD [s]
Single bay, negotiating	70	70	0	70	-0.4
Single bay, reactive	10	9	1	5	-1.5
Single bay, forced yielder	44	44	0	44	-0.5
Unequal urgency or speed	36	36	0	36	-0.6
Two bays	30	30	0	30	-0.2
Noisy bids ($\sigma = 0.5, 1$)	12	12	0	7	$+2.2$
Clarke misreports	18	18	0	18	-0.3
50% loss	18	18	0	18	-0.8
90% loss, 2 s latency	24	24	0	18	$+4.8$
Convoy (2 v 1)	36	36	0	36	-0.6
T-junction	36	36	0	35	-16.8
Different known bays	24	24	0	24	-0.3
Blocked bay	18	18	0	18	-1.2
Explore	84	84	0	83	$+0.2$

Succ.: success. Coll.: collisions. Same: same first yielder as 2D. ΔD: mean Gazebo minus 2D (urgency-weighted) total delay. The 50% loss runs also have 0.5 s latency.

G. Learned negotiator

On 150 held-out encounters in the training layout, the learned table reached rulebook-level weighted excess delay (+2.6 s, against +2.5 s for the rulebook, +9.2 s for fixed priority and 0.0 s for `cost_bid`). In an unseen single-bay layout it was no better than fixed priority (+13.1 s against +12.3 s), with 15% of observations unseen, so it does not generalize across layouts.

H. Gazebo validation

All Gazebo runs reported here use the final code: the 122 cases of earlier rounds (single-bay sweep, convoy, T-junction, 50% loss), rerun on the merged main branch, and 338 new cases on a commit that adds only Gazebo support (blocked-bay obstacles, per-run policy files) and explore mode. A Gazebo run takes about 5 min, so instead of the full 2D grids we ran every study's scenario on a reduced grid of 6–10 seeds per condition, each paired with a 2D run on the same seed (Table II). Extreme comms used the Waffle's 0.26 m/s rather than 0.5 m/s, and the protocol ablation was reduced to all safeguards on and all off. Seeds fix the start delays, but Gazebo physics and ROS timing are not deterministic, so each case is a single sample.

Across the 450 runs other than the reactive baseline (negotiating policies and forced yielders) there were no failures and no collisions (95% upper bound 0.85%), including the blocked bay, where touching the obstacle counts as a collision. 437 of the 450 chose the same first yielder as 2D. Of the 13 differences, 5 are noisy-bid runs, whose noise draw is keyed to the decision time, which differs between the simulators, 6 are 90%-loss runs, whose loss realizations differ, and one each is a T-junction and an explore run. Excess delay against each simulator's own best forced yielder transferred: for every policy in the single-bay, unequal-urgency and slow-robot cases it was within 1.0 s of the 2D value (e.g. fixed priority 7.5 vs 7.6 s, reservation 15.2 vs 15.2 s, learned 5.1 vs 5.3 s, `cost_bid` and Clarke 0 in both). In the other non-reactive groups, mean Gazebo delays were 0.2–1.2 s lower than in 2D; the exceptions are the T-junction, the noisy-bid and 90%-loss cases, where the differing draws dominate, and share-on-detect in explore mode (Sec. IV-F).

The T-junction transferred less well. When the branch robot leaves the branch, `cost_bid` finished 22 s earlier on average in Gazebo (11–75 s), and with fixed priority and the rulebook a second robot also yielded in 4 and 3 of 6 Gazebo runs, which never happened in 2D. Trace analysis on the earlier code attributed the `cost_bid` gap to a renegotiation stall in 2D; we have not re-traced these runs. When a robot turns into the branch, Gazebo makespans were 1–5 s shorter. An earlier Gazebo round, on older code, had exposed two sim-to-real gaps at the T-junction: a goal on the room wall was "reached" in 2D, which has no wall perception, and the branch walls occlude the lidar of a robot inside it. Both fixes are in the final code; they were found and checked on the same cases, so the T-junction rows are not an independent validation.

V. DISCUSSION AND LIMITATIONS

Three conclusions hold across our studies. First, communication is cheap insurance: any explicit agreement removes the deadlocks and back-off "dances" of the reactive baseline. Second, the right rule depends on what the robots know: cost bidding is best when robots differ and their costs are known to within $\sigma \approx 0.25$ in log terms, but it needs payments to resist unilateral misreporting and side-level costs to be efficient for convoys, and even our group bid trails fixed priority when the lone robot is urgent. Third, under extreme packet loss our protocol had no collisions, but the ablation shows that this cannot be credited to the commit step or the bay check alone.

The study has clear limits. All results are from simulation. The Gazebo runs cover a reduced grid with one nondeterministic run per case (6–10 seeds per condition), and physical TurtleBot3 trials are future work. `cost_bid` and the Clarke mechanism use the simulator's own motion model, so their efficiency and truthfulness hold relative to that model only, and the mechanism neither balances its budget nor resists collusion. The best forced yielder is a reference for the yield decision, not a global optimum, and only the convoy has a

three-robot reference. The T-junction is approximated as a deep bay, explore mode discovers bays geometrically rather than by SLAM, and the map studies use 10 seeds per cell. The learned baseline is tabular and centrally trained, not deep MARL, and we have not yet evaluated LLM negotiators, a natural next agent family for this benchmark [16], [17].

VI. Conclusion

We presented a benchmark for head-on corridor encounters with pull-in bays and compared negotiation policies under identical physics and communication models. Cost bidding with a good cost model matches the best forced yielder and handles asymmetric robots; Clarke-pivot payments remove the incentive for unilateral misreporting at the price of a budget surplus; a small fairness credit makes repeated encounters fair for 8% extra delay; and first-come reservation and fixed priority leave substantial delay on the table. The simulator, ROS 2 nodes, scenarios, study scripts, and per-episode results are released to support comparison with learned and language-model negotiators.

References

[1] J. Grover, C. Liu, and K. Sycara, "The Before, During, and After of Multi-robot Deadlock," *The International Journal of Robotics Research*, vol. 42, no. 6, pp. 317–336, 2022. [Online]. Available: https://doi.org/10.1177/02783649221074718

[2] S. Dergachev and K. Yakovlev, "Distributed Multi-Agent Navigation Based on Reciprocal Collision Avoidance and Locally Confined Multi-Agent Path Finding," in *2021 IEEE 17th International Conference on Automation Science and Engineering (CASE)*, 2021, pp. 1489–1494. [Online]. Available: https://doi.org/10.1109/case49439.2021.9551564

[3] R. Chandra, V. Zinage, E. Bakolas, P. Stone, and J. Biswas, "Deadlock-free, safe, and decentralized multi-robot navigation in social mini-games via discrete-time control barrier functions," *Autonomous Robots*, vol. 49, no. 2, 2025. [Online]. Available: https://doi.org/10.1007/s10514-025-10194-8

[4] M. Cap, J. Gregoire, and E. Frazzoli, "Provably safe and deadlock-free execution of multi-robot plans under delaying disturbances," in *2016 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2016, pp. 5113–5118. [Online]. Available: https://doi.org/10.1109/iros.2016.7759750

[5] K. Okumura, M. Machida, X. Défago, and Y. Tamura, "Priority inheritance with backtracking for iterative multi-agent path finding," *Artificial Intelligence*, vol. 310, p. 103752, 2022. [Online]. Available: https://doi.org/10.1016/j.artint.2022.103752

[6] M. Phillips and M. Likhachev, "SIPP: Safe interval path planning for dynamic environments," in *Proc. IEEE International Conference on Robotics and Automation (ICRA)*, 2011, pp. 5628–5635.

[7] F. Pratissoli, R. Brugioni, N. Battilani, and L. Sabattini, "Hierarchical Traffic Management of Multi-AGV Systems With Deadlock Prevention Applied to Industrial Environments," *IEEE Transactions on Automation Science and Engineering*, vol. 21, no. 3, pp. 3155–3169, 2024. [Online]. Available: https://doi.org/10.1109/tase.2023.3276233

[8] Open-RMF contributors, "rmf_traffic: Traffic Scheduling and Negotiation for Mobile Robot Fleets (software repository)," https://github.com/open-rmf/rmf_traffic, 2026. [Online]. Available: https://github.com/open-rmf/rmf_traffic

[9] Open Robotics, "Open-RMF: Open Robotics Middleware Framework," https://www.open-rmf.org/, 2026. [Online]. Available: https://www.open-rmf.org/

[10] S. Kato, S. Nishiyama, and J. Takeno, "Coordinating Mobile Robots By Applying Traffic Rules," in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems*, vol. 3, 1992, pp. 1535–1541. [Online]. Available: https://doi.org/10.1109/iros.1992.594218

[11] Y. Shoham and M. Tennenholtz, "On social laws for artificial agent societies: off-line design," *Artificial Intelligence*, vol. 73, no. 1-2, pp. 231–252, 1995. [Online]. Available: https://doi.org/10.1016/0004-3702(94)00007-n

[12] B. Gerkey and M. Mataric, "Sold!: auction methods for multirobot coordination," *IEEE Transactions on Robotics and Automation*, vol. 18, no. 5, pp. 758–768, 2002. [Online]. Available: https://doi.org/10.1109/tra.2002.803462

[13] A. Gautier, A. Stephens, B. Lacerda, N. Hawes, and M. Wooldridge, "Negotiated Path Planning for Non-Cooperative Multi-Robot Systems," in *International Joint Conference on Autonomous Agents and Multiagent Systems*, 2022, pp. 472–480. [Online]. Available: https://doi.org/10.65109/dvsq3214

[14] ProrokLab, "VMAS Scenario Source: give_way.py," https://github.com/proroklab/VectorizedMultiAgentSimulator/blob/main/vmas/scenarios/give_way.py, 2026. [Online]. Available: https://github.com/proroklab/VectorizedMultiAgentSimulator/blob/main/vmas/scenarios/give_way.py

[15] M. Bettini, A. Shankar, and A. Prorok, "Heterogeneous Multi-Robot Reinforcement Learning," in *International Joint Conference on Autonomous Agents and Multiagent Systems*, 2023, pp. 1485–1494. [Online]. Available: https://doi.org/10.65109/slob2372

[16] T. Godfrey, W. Hunt, and M. D. Soorati, "MARLIN: Multi-Agent Reinforcement Learning Guided by Language-Based Inter-Robot Negotiation," arXiv preprint, 2024. [Online]. Available: https://arxiv.org/abs/2410.14383

[17] V. Mahadevan, S. Zhang, and R. Chandra, "GameChat: Multi-LLM Dialogue for Safe, Agile, and Socially Optimal Multi-Agent Navigation in Constrained Environments," in *2025 IEEE International Symposium on Multi-Robot and Multi-Agent Systems (MRS)*, 2025, pp. 1–7. [Online]. Available: https://doi.org/10.1109/mrs66243.2025.11357267

[18] K. Garg, S. Zhang, J. Arkin, and C. Fan, "Foundation Models to the Rescue: Deadlock Resolution in Connected Multi-Robot Systems," arXiv preprint, 2024. [Online]. Available: https://arxiv.org/abs/2404.06413

[19] R. Chandra, S. Singh, W. Luo, and K. Sycara, "Multi-robot navigation in social mini-games: definitions, taxonomy, and algorithms," *Autonomous Robots*, vol. 50, no. 2, 2026. [Online]. Available: https://doi.org/10.1007/s10514-026-10251-w

[20] R. Stern, N. Sturtevant, A. Felner, S. Koenig, H. Ma, T. Walker, J. Li, D. Atzmon, L. Cohen, T. K. Kumar, R. Barták, and E. Boyarski, "Multi-Agent Pathfinding: Definitions, Variants, and Benchmarks," *Proceedings of the International Symposium on Combinatorial Search*, vol. 10, no. 1, pp. 151–158, 2021. [Online]. Available: https://doi.org/10.1609/socs.v10i1.18510

[21] R. Moan, C. McBeth, M. Morales, N. Amato, and K. Hauser, "Experience-based multi-agent path finding with narrow corridors," in *Robotics: Science and Systems XX*, 2024. [Online]. Available: https://doi.org/10.15607/rss.2024.xx.087

[22] A. Pertzovskiy, R. Stern, R. Zivan, and A. Felner, "Multi-Agent Corridor Generating Algorithm," in *Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence*, 2025, pp. 240–247. [Online]. Available: https://doi.org/10.24963/ijcai.2025/28

[23] A. Mannucci, L. Pallottino, and F. Pecora, "Provably Safe Multi-Robot Coordination With Unreliable Communication," *IEEE Robotics and Automation Letters*, vol. 4, no. 4, pp. 3232–3239, 2019. [Online]. Available: https://doi.org/10.1109/lra.2019.2924849

[24] W. Vickrey, "COUNTERSPECULATION, AUCTIONS, AND COMPETITIVE SEALED TENDERS," *The Journal of Finance*, vol. 16, no. 1, pp. 8–37, 1961. [Online]. Available: https://doi.org/10.1111/j.1540-6261.1961.tb02789.x

[25] E. H. Clarke, "Multipart pricing of public goods," *Public Choice*, vol. 11, no. 1, pp. 17–33, 1971.

[26] T. Groves, "Incentives in teams," *Econometrica*, vol. 41, no. 4, pp. 617–631, 1973.

[27] N. Nisan and A. Ronen, "Algorithmic Mechanism Design," *Games and Economic Behavior*, vol. 35, no. 1-2, pp. 166–196, 2001. [Online]. Available: https://doi.org/10.1006/game.1999.0790

[28] J. Green and J.-J. Laffont, "Characterization of satisfactory mechanisms for the revelation of preferences for public goods," *Econometrica*, vol. 45, no. 2, pp. 427–438, 1977.

[29] R. B. Myerson and M. A. Satterthwaite, "Efficient mechanisms for bilateral trading," *Journal of Economic Theory*, vol. 29, no. 2, pp. 265–281, 1983.

[30] A. Censi, S. Bolognani, J. G. Zilly, S. S. Mousavi, and E. Frazzoli, "Today me, tomorrow thee: Efficient resource allocation in competitive settings using karma games," in *Proc. IEEE Intelligent Transportation Systems Conference (ITSC)*, 2019, pp. 686–693.

[31] E. Elokda, S. Bolognani, A. Censi, F. Dörfler, and E. Frazzoli, "A self-contained karma economy for the dynamic allocation of common resources," *Dynamic Games and Applications*, vol. 14, no. 3, pp. 578–610, 2024.

[32] M. Baumann, J. Beyerer, H. Buck, B. Deml, S. Ehrhardt, C. Frese, D. Kleiser, M. Lauer, M. Roschani, M. Ruf, C. Stiller, P. Vortisch, and J. Ziehn, "Cooperative Automated Driving for Bottleneck Scenarios in Mixed Traffic," in *2023 IEEE Intelligent Vehicles Symposium (IV)*, 2023, pp. 1–8. [Online]. Available: https://doi.org/10.1109/iv55152.2023.10186638

[33] R. Jain, D.-M. Chiu, and W. R. Hawe, "A Quantitative Measure of Fairness and Discrimination for Resource Allocation in Shared Computer Systems," Digital Equipment Corporation, Tech. Rep. DEC-TR-301, 1984. [Online]. Available: https://arxiv.org/abs/cs/9809099